LECTURE-30

Polymorphism:

Introduction

When an object is created from its class, the member variables and member functions are allocated memory spaces. The memory spaces have unique addresses. Pointer is a mechanism to access these memory locations using their address rather than the name assigned to them. You will study the implications and applications of this mechanism in detail in this chapter.

Pointer is a variable which can hold the address of a memory location rather than the value at the location. Consider the following statement

```
int num =84;
```

This statement instructs the compiler to reserve a 2-byte of memory location and puts the value 84 in that location. Assume that the compiler allocates memory location 1001 to num. Diagrammatically, the allocation can be shown as:

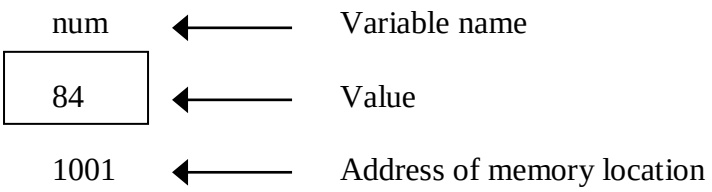


Figure 9.1

As the memory addresses are themselves numbers, they can be assigned to some other variable For example, ptr be the variable to hold the address of variable num.

Thus, we can access the value of num by the variable ptr. We can say "ptr points to num" as shown in the figure below.

Fig 9.2

Pointers to Objects

An object of a class behaves identically as any other variable. Just as pointers can be defined in case of base C++ variables so also pointers can be defined for an object type. To create a pointer variable for the following class

```
class employee   {
        int code;
        char name   [20] ;
      public:
        inline void getdata ( )= 0 ;
        inline void display ( )= 0 ;
};
```

The following codes is written

```
employee  *abc;
```

This declaration creates a pointer variable abc that can point to any object of employee type.

this Pointer

C++ uses a unique keyword called "this" to represent an object that invokes a member function. 'this' is a pointer that points to the object for which this function was called. This unique pointer is called and it passes to the member function automatically. The pointer this acts as an implicit argument to all the member function, for e.g.

```
class ABC
{
  int a ;
      -----
      -----
 };
```

The private variable ‘a’ can be used directly inside a member function, like

```
a=123;
```

We can also use the following statement to do the same job.

```
this → a = 123
```

e.g.

```
class  stud
{
      int a;
      public:
      void set   (int a)
      {
      this  →  a  =  a;   //here   this  point   is  used  to assign a  class   level
      }          ‘a’   with the argument   ‘a’
      void show   (   )
      {
      cout  <<  a;
      }
};
main   (   )
{
      stud S1, S2;
```

```
        S1.bet   (5) ;
        S2.show   (   );
}
o/p =  5
```

Pointers to Derived Classes

Polymorphism is also accomplished using pointers in C++. It allows a pointer in a base class to point to either a base class object or to any derived class object. We can have the following Program segment show how we can assign a pointer to point to the object of the derived class.

```
class base
{
        //Data Members
        //Member Functions
};
class derived : public base
{
        //Data Members
        //Member functions
};

void main   (   ) {
        base *ptr;   //pointer to class base
        derived obj ;
        ptr =  &obj ;      //indirect  reference obj   to the pointer
        //Other Program statements

}
```

The pointer ptr points to an object of the derived class obj. But, a pointer to a derived class object may not point to a base class object without explicit casting.

For example, the following assignment statements are not valid

```
void main   (   )
{
        base  obja;
        derived *ptr;
        ptr =  &obja;   //invalid.... .explicit  casting required
        //Other Program statements
}
```

A derived class pointer cannot point to base class objects. But, it is possible by using explicit casting.

```
void main   (   )
{
        base obj ;
        derived *ptr;            // pointer of  the derived class
        ptr =   (derived *)   & obj;            //correct  reference
        //Other Program statements
}
```

Student Activity

1. Define Pointers.
2. What are the various operators of pointer? Describe their usage.
3. How will you declare a pointer in C++?

LECTURE-31

Virtual Functions

Virtual functions, one of advanced features of OOP is one that does not really exist but it« appears real in some parts of a program. This section deals with the polymorphic features which are incorporated using the virtual functions.

The general syntax of the virtual function declaration is:

```
class use_detined_name{
private:
public:
virtual return_type function_name1 (arguments);
virtual return_type function_name2(arguments);
virtual return_type function_name3( arguments);
----------
};
```

To make a member function virtual, the keyword virtual is used in the methods while it is declared in the class definition but not in the member function definition. The keyword virtual precedes the return type of the function name. The compiler gets information from the keyword virtual that it is a virtual function and not a conventional function declaration.

For. example, the following declararion of the virtual function is valid.

```
class point {
intx;
inty;
public:
virtual int length ( );
virtual void display ( );
};
```

Remember that the keyword virtual should not be repeated in the definition if the definition occurs outside the class declaration. The use of a function specifier virtual in the function definition is invalid.

For example

```
class point   {
intx ;
inty ;
public:
virtual void display   ( );
};
virtual void point: : display ( ) //error
{
Function Body
}
```

A virtual function cannot be a static member since a virtual member is always a member of a particular object in a class rather than a member of the class as a whole.

```
class point   {
int x ;
int y ;
public:
virtual  static  int  length   ( );   //error
```

```
};
        int point: : length ( )
        {
        Function body
        }
```

A virtual function cannot have a constructor member function but it can have the destructor member function.

```
class point {
int x ;
int y ;
public:
virtual point (int xx, int yy) ; // constructors, error
void display ( ) ;
int length ( ) ;
};
```

A destructor member function does not take any argument and no return type can be specified for it not even void.

```
class point   {
int x ;
int y ;
public:
virtual  point   (int xx,   int yy) ;   //invalid
void display   (   ) ;
int  length   (   ) ;
```

It is an error to redefine a virtual method with a change of return data type in the derived class with the same parameter types as those of a virtuall method in the base class.

```
class base {
int x,y ;
public:
virtual int sum (int xx, int yy ) ; //error
} ;
class derived: public base {
intz ;
public:
virtual float sum (int xx, int yy) ;
};
```

The above declarations of two virtual functions are invalid. Even though these functions take identical arguments note that the return data types are different.

```
virtual  int  sum   (int xx,   int  IT) ;   //base  class
virtual  float  sum   (int xx,   int  IT) ;   //derived class
```

Both the above functions can be written with int data types in the base class as well as in the derived class as

```
virtual  int  sum   (int xx,   int yy) ;   //base class
virtual  int  sum   (int xx,   int yy) ;   //derived class
```

Only a member function of a class can be declared as virtual. A non member function (nonmethod) of a class cannot be declared virtual.

```
virtual void display   ( )   //error,   nonmember function
{
        Function body
}
```

Student Activity

1. What are virtual functions
2. What are pure virtual functions
3. Define Virtual destructors.

Late Binding

As we studied in the earlier unit, late binding means selecting functions during the execution. Though late binding requires some overhead it provides increased power and flexibility. The late binding is implemented through virtual functions as a result we have to declare an object of a class either as a pointer to a class or a reference to a class.

For example the following shows how a late binding or run time binding can be carried out with the help of a virtual function.

```
class base {
private :
int x;
float y;
public:
virtual void display ( ) ;
int sum ( ) ;
};
class derivedD : public baseA
{
private :
int x ;
float y;
public:
void display ( ); //virtual
int sum ( ) ;
};
void main ( )
{
        baseA *ptr ;
        derivedD objd ;
        ptr = &objd ;
        Other Program statements
        ptr- >di splay   ( ) ;   //run time binding
        ptr->sum   (   ) ;   //compile  time binding
}
```

Note that the keyword virtual is be followed by the return type of a member function if a run time is to be bound. Otherwise, the compile time binding will be effected as usual. In the above program segment, only the display ( ) function has been declared as virtual in the base class, whereas the sum ( ) is nonvirtual. Even though the message is given from the pointer of the base class to the objects of the derived class, it will not

access the sum ( ) function of the derived class as it has been declared as nonvirtual. The sum ( ) function compiles only the static binding.

The following program demonstrates the run time binding of the member functions of a class. The same message is given to access the derived class member functions from the array of pointers. As function are declared as virtual, the C++ compiler invokes the dynamic binding.

```
#include <iostream.h>
#include <conio.h>
class baseA {
public :
virtual void display () {
cout<< “One \n”;
}
};
class derivedB : public baseA
{
        public:
        virtual void display(){
        cout<< “Two\n”; }
};
class derivedC: public derivedB
{
        public:
        virtual void display ( ) {
        cout<< “Three \n”; }
};
void main ( ) {
        //define three objects
        baseA obja;
        derivedB objb;
        derivedC objc;
        base A *ptr   [3];   //define  an array of pointers  to baseA
        ptr   [0]   =   &obja;
        ptr   [1]   =  &objb;
        ptr   [2]   =  &objc;
        for   ( int  i  =  0;   i  <=2;   i  ++   )
        ptr   [i]->display   ( );   //same  message  for  all   objects
        getche   (   ) ;
}
```

Output
One
Two
Three

The program listed below illustrates the static binding of the member functions of a class. In program there are two classes student and academic. The class academic is derived from class student. The two member function getdata and display are defined for both the classes. *obj is defined for class student, the address of which is stored in the object of the class academic. The functions getdata ( ) and display ( ) of student class are invoked by the pointer to the class.

```
#include<iostream.h>
#include<conio.h>
class student {
private:
int rollno;
char name [20];
public:
void getdata ( );
void display ( );
```

```
};
class academic: public student {
private:
char stream;
public:
void getdata ( );
void display ( ) ;
};
void student:: getdata ( )
{
        cout<< "enterrollno\n";
        cin>>rollno;
        cout<< "enter   name \n";
        cin>>name;
}
void student:: display ( )
{
cout<< "the student's roll number is "<<rollno<< "and name is"<<name ;
cout<< endl;
}
void academic :: getdata ( )
{
        cout<< "enter stream of a student? \n";
        cin >>stream;
}
void academic :: display ( ) {
        cout<< "students stream \n";
        cout <<stream<< endl;
}
void main ( )
{
        student  *ptr ;
        academic obj ;
        ptr=&obj;
        ptr->getdata ( ) ;
        ptr->display ( ) ;
        getche   (   );
}
```

output

enter rollno

25

enter name

raghu

the student's roll number is 25 and name is raghu

The program listed below illustrates the dynamic binding of member functions of a class. In this program there are two classes student and academic. The class academic is derived from student. Student function has two virtual functions getdata ( ) and display (). The pointer for student class is defined and object . for academic class is created. The pointer is assigned the address of the object and function of derived class are invoked by pointer to student.

```
#include <iostream.h>
#include <conio.h>
class student {
```

```
private:
introllno;
char name [20];
public:
virtual void getdata ( );
virtual void display ( );
};
class academic: public student {
private :
char stream[10];
public:
void getdata { };
void display ( ) ;
};
void student: : getdata ( )
{
        cout<< “enter rollno\n”;
        cin >> rollno;
        cout<< “enter name \n”;
        cin >>name;
}
void student:: display ( )
{
cout<< “the student’s roll number is”<<rollno<< “and name is”<<name;
cout<< end1;
}
void academic: : getdata ( )
{
        cout << “enter stream of a student? \n”;
        cin>> stream;
}
void academic:: display ( )
{
        cout<< “students stream \n”;
        cout<< stream << endl;
}
void main ( )
{
        student  *ptr ;
        academic  obj ;
        ptr =  &obj ;
        ptr->getdata ( );
        ptr->dlsplay ( );
        getch ( );
}
```

output

enter stream of a student?

Btech

students stream

Btech

LECTURE-32

Pure Virtual Functions

Generally a function is declared virtual inside a base class and we redefine it the derived classes. The function declared in the base class seldom performs any task.

The following program demonstrates how a pure virtual function is defined, declared and invoked from the object of a derived class through the pointer of the base class. In the example there are two classes employee and grade. The class employee is base class and the grade is derived class. The functions getdata ( ) and display ( ) are declared for both the classes. For the class employee the functions are defined with empty body or no code inside the function. The code is written for the grade class. The methods of the derived class are invoked by the pointer to the base class.

```
#include<iostream.h>
#include<conio.h>
class employee {
int code
char name [20] ;
public:
virtual void getdata ( ) ;
virtual void display ( ) ;
};
class grade: public employee
{
        char grd [90] ;
        float salary ;
public :
        void getdata ( ) ;
        void display ( );
};
void employee :: getdata ( )
{
}
void employee:: display ( )
{
}
void grade : : getdata ( )
{
        cout<< " enter employee's grade ";
        cin> > grd ;
        cout<< "\n enter the salary " ;
        cin>> salary;
}
void grade : : display ( )
{
        cout«" Grade salary \n";
        cout« grd« " "« salary« endl;
```

```
}
void main ( )
{
        employee *ptr ;
        grade obj ;
        ptr = &obj ;
        ptr->getdata ( ) ;
        ptr->display ( ) ;
        getche ( ) ;
}
```

Output
enter employee's grade A
enter the salary 250000
Grade salary
A 250000

Object Slicing:

In C++, a derived class object can be assigned to a base class object, but the other way is not possible.

```
class Base { int x, y; };

class Derived : public Base { int z, w; };

int main()
{
    Derived d;
    Base b = d; // Object Slicing,  z and w of d are sliced off
}
```

Object Slicing happens when a derived class object is assigned to a base class object, additional attributes of a derived class object are sliced off to form the base class object.

```
#include <iostream>
using namespace std;

class Base
{
protected:
    int i;
public:
    Base(int a)     { i = a; }
    virtual void display()
    { cout << "I am Base class object, i = " << i << endl; }
};

class Derived : public Base
{
    int j;
public:
    Derived(int a, int b) : Base(a) { j = b; }
    virtual void display()
    { cout << "I am Derived class object, i = "
           << i << ", j = " << j << endl;  }
};
```

```
// Global method, Base class object is passed by value
void somefunc (Base obj)
{
    obj.display();
}

int main()
{
    Base b(33);
    Derived d(45, 54);
    somefunc(b);
    somefunc(d);  // Object Slicing, the member j of d is sliced off
    return 0;
}
```

Output:

```
I am Base class object, i = 33
I am Base class object, i = 45
```

We can avoid above unexpected behavior with the use of pointers or references. Object slicing doesn't occur when pointers or references to objects are passed as function arguments since a pointer or reference of any type takes same amount of memory. For example, if we change the global method myfunc() in the above program to following, object slicing doesn't happen.

```
// rest of code is similar to above
void somefunc (Base &obj)
{
    obj.display();
}
// rest of code is similar to above
```

Output:

```
I am Base class object, i = 33
I am Derived class object, i = 45, j = 54
```

We get the same output if we use pointers and change the program to following.

```
// rest of code is similar to above
void somefunc (Base *objp)
{
    objp->display();
}
int main()
{
    Base *bp = new Base(33) ;
    Derived *dp = new Derived(45, 54);
    somefunc(bp);
    somefunc(dp);  // No Object Slicing
    return 0;
}
```

Output:

```
I am Base class object, i = 33
I am Derived class object, i = 45, j = 54
```

Object slicing can be prevented by making the base class function pure virtual there by disallowing object creation. It is not possible to create the object of a class which contains a pure virtual method.

LECTURE-33

C++ Function Overriding

If base class and derived class have member functions with same name and arguments. If you create an object of derived class and write code to access that member function then, the member function in derived class is only invoked, i.e., the member function of derived class overrides the member function of base class. This feature in C++ programming is known as function overriding.

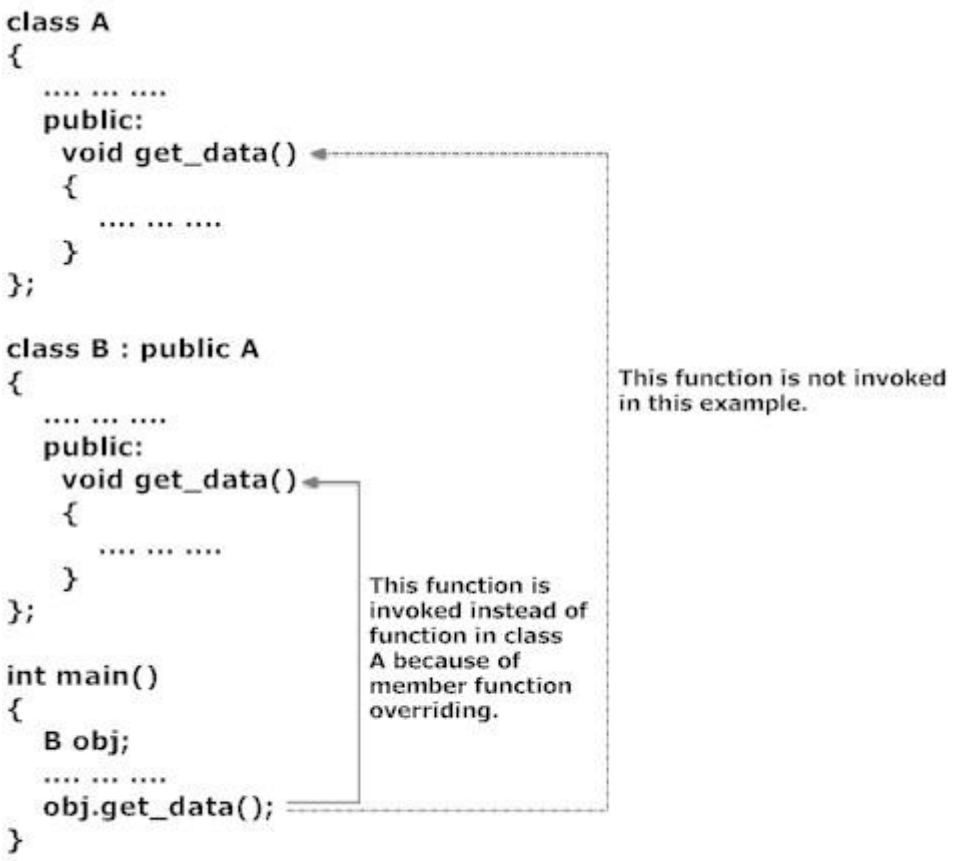


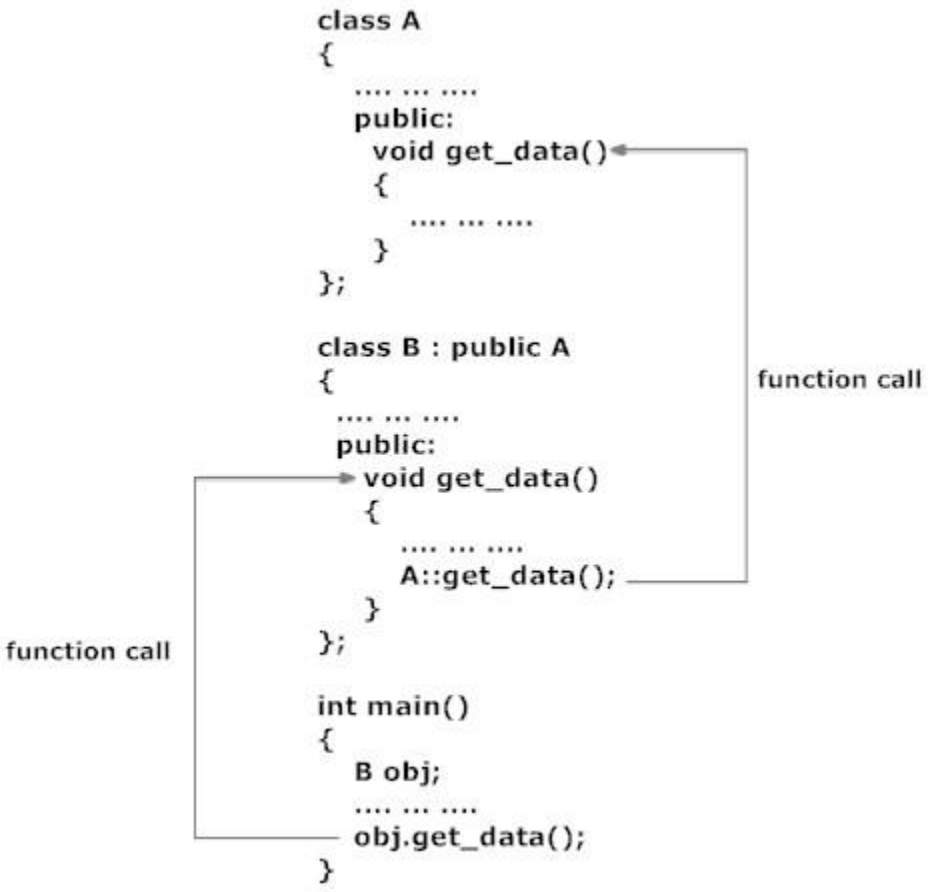
Figure: Member Function Overriding in C++

Accessing the Overridden Function in Base Class From Derived Class

To access the overridden function of base class from derived class, scope resolution operator **::**. For example: If you want to access `get_data()` function of base class from derived class in above example then, the following statement is used in derived class.

```
A::get_data; // Calling get_data() of class A.
```

It is because, if the name of class is not specified, the compiler thinks `get_data()` function is calling itself.



Abstract Class

Abstract Class is a class which contains atleast one Pure Virtual function in it. Abstract classes are used to provide an Interface for its sub classes. Classes inheriting an Abstract Class must provide definition to the pure virtual function, otherwise they will also become abstract class.

Characteristics of Abstract Class

1. Abstract class cannot be instantiated, but pointers and refrences of Abstract class type can be created.
2. Abstract class can have normal functions and variables along with a pure virtual function.
3. Abstract classes are mainly used for Upcasting, so that its derived classes can use its interface.
4. Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.

Pure Virtual Functions

Pure virtual Functions are virtual functions with no definition. They start with **virtual** keyword and ends with = 0. Here is the syntax for a pure virtual function,

virtual void f() = 0;

Example of Abstract Class

class Base //Abstract base class

```
{
 public:
 virtual void show() = 0;          //Pure Virtual Function
};

class Derived:public Base
{
 public:
 void show()
 { cout << "Implementation of Virtual Function in Derived class"; }
};

int main()
{
 Base obj;      //Compile Time Error
 Base *b;
 Derived d;
 b = &d;
 b->show();
}
```

Output : Implementation of Virtual Function in Derived class

In the above example Base class is abstract, with pure virtual **show()** function, hence we cannot create object of base class.

Why can't we create Object of Abstract Class ?

When we create a pure virtual function in Abstract class, we reserve a slot for a function in the VTABLE(studied in last topic), but doesn't put any address in that slot. Hence the VTABLE will be incomplete.

As the VTABLE for Abstract class is incomplete, hence the compiler will not let the creation of object for such class and will display an errror message whenever you try to do so.